MINISTRY OF EDUCATION AND TRAINING

FPT UNIVERSITY

Capstone Project Document

An AI Agent capable of autonomous decision-making for network defense within a simulation environment

Group Member	Student's Name	Student's ID
Student 1	Hồ Lê Bình	SE183564
Student 2	Nguyễn Hoàng Trí	SE183413
Student 3	Phạm Tuấn Anh	SE183403
Student 4	Trịnh Nguyễn Yến Vy	SE183776

Supervisor: Mai Hoàng Đỉnh

Hồ Chí Minh, 01/2026

ABSTRACT

The rapid automation and scalability of modern cyber attacks have rendered traditional rule-based security mechanisms increasingly ineffective. In the contemporary cybersecurity landscape, threats are evolving with unprecedented complexity and automation. Traditional defense mechanisms, primarily rule-based firewalls and static Intrusion Detection Systems (IDS), often fail to counter novel zero-day attacks or adapt to dynamic adversary tactics in real-time.

This study addresses these limitations by developing an Adaptive AI Defense Agent utilizing Reinforcement Learning (RL). Unlike traditional supervised learning, the proposed RL agent learns optimal defense strategies through autonomous interaction with a simulated environment via a trial-and-error mechanism. The project implements a customized network topology using OpenAI Gymnasium and Mininet, where the agent is trained to observe network states and execute enforcement actions—such as rate limiting, blocking, and honeypot redirection—to maximize system security while maintaining service availability.

Performance is evaluated against multi-vector attack baselines, with results visualized through Wazuh SIEM. The findings demonstrate that autonomous AI agents can significantly enhance the resilience of network infrastructures against adaptive threats. This work contributes a reproducible simulation framework and demonstrates the feasibility of reinforcement learning-based autonomous network defense.

Contents

Chapter 1

INTRODUCTION

1.1 Background

The cybersecurity landscape has undergone a significant transformation over the past decade, driven by the rapid growth of digital infrastructures, cloud computing, and large-scale interconnected networks. Alongside these developments, cyber threats have become increasingly automated, adaptive, and sophisticated, posing serious challenges to traditional security mechanisms.

1.1.1 Escalation of Cyber Threats and Attack Automation

Modern cyber attacks are no longer predominantly manual or opportunistic. Instead, adversaries increasingly rely on automated tools to perform large-scale reconnaissance, vulnerability scanning, credential stuffing, and exploitation at machine speed. According to the Verizon Data Breach Investigations Report (DBIR), approximately 60% of confirmed data breaches involve the human element, including social engineering, credential misuse, and user error, while the exploitation of software vulnerabilities and web applications continues to rise [1].

Furthermore, web application attacks and system intrusions remain among the most prevalent breach patterns. These attacks are often highly automated, enabling threat actors to rapidly identify exposed services and exploit weaknesses across thousands of targets simultaneously. Such large-scale automation significantly reduces the effectiveness of static, rule-based security systems that rely on predefined signatures or manually crafted rules [2].

In addition to external adversaries, insider-related incidents represent a substantial portion of cybersecurity breaches. Earlier DBIR findings indicate that nearly 30% of data breaches involved internal actors, either through malicious intent or unintentional actions, highlighting that threats can originate from within organizational boundaries as well as

from external attackers [3]. This diversity of threat actors further complicates the defense landscape and demands more adaptive security strategies.

1.1.2 Economic Impact of Data Breaches

Beyond technical consequences, cyber attacks impose severe financial and operational costs on organizations. According to the IBM–Ponemon Institute Cost of a Data Breach Report, the global average cost of a data breach has reached several million US dollars per incident, with significantly higher costs observed in highly regulated industries and large enterprises [4]. These costs include incident response, system recovery, legal penalties, reputational damage, and long-term loss of customer trust.

The increasing frequency and scale of cyber incidents, combined with their substantial economic impact, underscore the necessity for faster detection and response mechanisms. Delayed or ineffective responses can dramatically amplify financial losses, making real-time or near-real-time defense capabilities a critical requirement for modern network infrastructures.

1.1.3 Evolution of Network Infrastructure and Operational Challenges

The evolution from traditional on-premise hardware toward virtualized and cloud-based infrastructures has further expanded the attack surface. Technologies such as virtualization, containerization, and software-defined networking enable flexible and scalable deployments, but they also generate massive volumes of network traffic and security events. As a result, Security Operations Centers (SOCs) are frequently overwhelmed by an excessive number of alerts, a phenomenon commonly referred to as *alert fatigue* [5].

In such environments, human analysts struggle to manually inspect and correlate alerts in a timely manner. Critical security incidents may be overlooked or detected too late, allowing attackers to persist within networks for extended periods. This operational bottleneck exposes the limitations of purely human-driven security monitoring in large-scale, high-speed network environments.

1.1.4 Need for Autonomous and Adaptive Defense Systems

Given the automation of cyber attacks, the economic consequences of breaches, and the complexity of modern network infrastructures, there is a growing need for autonomous and adaptive defense mechanisms. Artificial Intelligence (AI), particularly Reinforcement Learning (RL), has emerged as a promising approach to address these challenges [6].

Unlike traditional supervised learning methods that rely on labeled datasets, RL enables agents to learn optimal defense strategies through continuous interaction with the envi-

ronment. By observing network states and receiving feedback in the form of rewards or penalties, an RL-based defense agent can dynamically adapt its behavior to evolving attack patterns. This capability makes RL especially suitable for real-time network defense scenarios, where attack strategies and system conditions change rapidly.

Consequently, the integration of autonomous AI agents into network defense architectures represents a critical step toward enhancing the resilience and responsiveness of cybersecurity systems.

1.2 Problem Statement

Despite significant advances in cybersecurity technologies, most existing network defense mechanisms remain largely reactive and heavily dependent on predefined rules and human intervention. Traditional security solutions, such as rule-based firewalls, signature-based Intrusion Detection Systems (IDS), and manually configured access control policies, are effective only against known attack patterns. These systems struggle to detect zero-day attacks, adapt to evolving adversarial behaviors, or respond at the speed required to counter highly automated threats.

Furthermore, modern network environments generate massive volumes of security-related data due to the adoption of cloud computing, virtualization, and software-defined networking. Security Operations Centers (SOCs) are often overwhelmed by continuous streams of alerts, leading to delayed responses and an increased risk of alert fatigue. As a result, human analysts may overlook critical threats or react too slowly, allowing attackers to maintain persistence within the network.

Another key limitation of existing defense approaches is the lack of autonomous decision-making capabilities. Most security systems rely on static policies or require manual tuning by administrators, making them unsuitable for dynamic and adversarial environments where attack strategies change rapidly. Even machine learning-based solutions frequently depend on supervised learning techniques that require large, labeled datasets, which are difficult to obtain and quickly become outdated in real-world cybersecurity scenarios.

Given these challenges, there is a clear need for an adaptive defense mechanism that can autonomously observe network conditions, identify malicious behaviors, and execute appropriate mitigation actions in real time. Such a system must be capable of learning from interaction with the environment, balancing security objectives with service availability, and continuously adapting to new attack patterns without requiring explicit prior knowledge of all possible threats.

This project addresses these limitations by proposing an AI-driven network defense agent based on Reinforcement Learning (RL). By formulating network defense as a sequential decision-making problem, the RL agent is designed to learn optimal response strategies

through trial-and-error interactions within a simulated network environment. The central problem investigated in this study is how to design, train, and evaluate an autonomous RL-based defense agent that can effectively mitigate diverse cyber attacks while maintaining acceptable network performance.

1.3 Research Objectives

The primary objective of this project is to design, implement, and evaluate an autonomous AI-based network defense system capable of making real-time security decisions in a dynamic and adversarial environment. To achieve this overarching goal, the study is guided by the following specific research objectives:

- Investigate the applicability of Reinforcement Learning (RL) techniques for autonomous decision-making in network defense scenarios, particularly in environments characterized by automated and adaptive cyber attacks.
- Design and construct a simulated network environment that accurately represents real-world network infrastructures, including production servers, attackers, and defensive components, using the Gymnasium framework and Mininet.
- Develop an RL-based defense agent capable of observing network states, identifying anomalous or malicious behaviors, and selecting appropriate mitigation actions such as blocking, rate limiting, or traffic redirection.
- Define and implement a reward function that balances security effectiveness with system performance, ensuring that defensive actions mitigate attacks while preserving service availability.
- Evaluate the performance of the proposed defense agent against multiple attack scenarios by measuring key metrics such as attack mitigation rate, response time, false positive rate, and overall network stability.

By addressing these objectives, the project aims to demonstrate the feasibility and effectiveness of Reinforcement Learning as a foundation for autonomous network defense systems and to provide empirical insights into their behavior under diverse attack conditions.

1.4 Scope and Limitations

This section defines the scope of the research and clarifies the limitations of the proposed approach in order to establish realistic expectations for the outcomes of the study.

1.4.1 Scope of the Study

The scope of this project is focused on the design and evaluation of an autonomous network defense agent within a controlled simulation environment. The research emphasizes decision-making at the network level rather than application-level security or user behavior analysis.

Attack Simulation

The simulated environment incorporates multiple categories of cyber attacks inspired by the MITRE ATT&CK framework [7]. These attacks are selected to represent common and impactful threat vectors observed in real-world networks and include:

- Denial-of-Service (DoS) and Distributed Denial-of-Service (DDoS) attacks, such as TCP SYN flood, UDP flood, and HTTP flood attacks.
- Reconnaissance activities, including port scanning and service enumeration.
- Web and authentication-based attacks, such as brute-force login attempts and SQL injection.

Defense Mechanisms

The defense agent is designed to execute mitigation actions at the operating system and network layer. These actions include:

- [**ALLOW**] Default action: allowing normal traffic to pass through without intervention.
- [**BLOCK**] Blocking malicious IP addresses using firewall rules.
- [**RATE**] Rate limiting suspicious traffic to reduce attack impact while preserving legitimate services.
- [**REDIRECT**] Redirecting selected traffic to honeypots to isolate attackers and gather intelligence.

All defensive actions are implemented using Linux-based mechanisms, such as `iptables` and traffic control (`tc`), within the simulated network.

Technology Stack

The implementation leverages a set of widely adopted open-source tools and frameworks, including:

- Python as the primary programming language.
- Gymnasium for defining the reinforcement learning environment.

- Mininet for emulating virtual network topologies.
- PyTorch and Stable Baselines3 for training reinforcement learning models.
- Wazuh SIEM for log collection, visualization, and performance monitoring.

1.4.2 Limitations of the Study

Despite its contributions, this study has several limitations that should be acknowledged.

First, the proposed defense system is evaluated exclusively within a simulated environment. Although the simulation is designed to approximate real-world network behavior, results may differ when deployed in production environments due to factors such as hardware constraints, network heterogeneity, and unpredictable user behavior.

Second, the attack scenarios considered in this research are limited to a predefined set of attack types. Advanced threats such as zero-day exploits, supply chain attacks, and sophisticated lateral movement techniques are outside the scope of this study.

Third, the performance of the reinforcement learning agent is influenced by the quality of the reward function and the representativeness of the simulated environment. Suboptimal reward design or incomplete state representation may limit the agent's ability to generalize to unseen scenarios.

Finally, the system is implemented and tested on Ubuntu Linux platforms. The portability of the proposed approach to other operating systems or large-scale distributed cloud environments is not evaluated in this work and remains a topic for future research.

1.5 Significance of the Study

This study contributes to the field of cybersecurity by exploring the feasibility of autonomous network defense using Reinforcement Learning. By formulating network defense as a sequential decision-making problem, the proposed approach advances beyond traditional rule-based and reactive security mechanisms toward adaptive and self-learning defense systems.

From a technical perspective, the project provides a practical implementation of an RL-based defense agent integrated with a simulated network environment. The combination of Gymnasium, Mininet, and Linux-based mitigation techniques demonstrates how reinforcement learning can be applied to real-world-inspired network scenarios. The integration with Wazuh SIEM further illustrates the potential for combining autonomous defense agents with existing security monitoring platforms to enhance visibility and operational awareness.

From an academic standpoint, this research offers empirical insights into the behavior of reinforcement learning agents under diverse cyber attack scenarios. The evaluation

results contribute to a better understanding of how autonomous agents balance security effectiveness with service availability, a critical trade-off in network defense. The findings may serve as a reference for future studies investigating adaptive cybersecurity systems and reinforcement learning-based decision-making.

Finally, from a practical and industrial perspective, the proposed framework highlights the potential of AI-driven autonomous defense systems to reduce the operational burden on human security analysts. By automating detection and response processes, such systems can mitigate alert fatigue and enable faster, more consistent reactions to cyber threats. This study thus lays foundational groundwork for future research and development of intelligent, scalable, and resilient network defense solutions.

Chapter 2

LITERATURE REVIEW

2.1 Reinforcement Learning Fundamentals

Reinforcement Learning (RL) is a branch of machine learning that focuses on enabling agents to learn optimal behaviors through interaction with an environment. Unlike supervised learning, which relies on labeled datasets, RL allows an agent to learn directly from experience by receiving feedback in the form of rewards or penalties. This learning paradigm is particularly suitable for dynamic and adversarial domains such as cybersecurity, where labeled data is scarce and attack strategies evolve continuously.

2.1.1 Reinforcement Learning Framework

The reinforcement learning process is typically modeled as an interaction between an agent and an environment. At each discrete time step, the agent observes the current state of the environment and selects an action according to its policy. In response, the environment transitions to a new state and provides a scalar reward that reflects the quality of the agent's action.

This interaction is commonly formalized using a Markov Decision Process (MDP), defined by a tuple (S, A, P, R, γ), where:

- S represents the set of possible states.
- A denotes the set of available actions.
- $P(s'|s, a)$ defines the state transition probability.
- $R(s, a)$ is the reward function.
- $\gamma \in [0, 1]$ is the discount factor that balances immediate and future rewards.

The objective of the agent is to learn a policy $\pi(a|s)$ that maximizes the expected cumulative reward over time. In the context of network defense, this corresponds to learning

sequences of mitigation actions that reduce attack impact while maintaining acceptable system performance.

2.1.2 Model-Free and Model-Based Reinforcement Learning

Reinforcement learning algorithms can be broadly categorized into model-based and model-free approaches. Model-based RL methods attempt to learn or utilize an explicit model of the environment's dynamics, which can then be used for planning and decision-making. While such methods can be sample-efficient, accurately modeling complex network environments and attacker behaviors is often impractical.

Model-free RL, on the other hand, learns optimal policies directly from interactions without requiring an explicit model of the environment. These methods are more commonly applied in cybersecurity research due to their flexibility and ability to handle complex, partially observable environments. Popular model-free approaches include value-based methods and policy-based methods.

2.1.3 Deep Reinforcement Learning

Traditional reinforcement learning techniques struggle to scale to environments with large or continuous state spaces. Deep Reinforcement Learning (DRL) addresses this limitation by integrating deep neural networks as function approximators for value functions or policies.

Value-based DRL methods, such as Deep Q-Networks (DQN), approximate the action-value function using neural networks and select actions that maximize expected rewards. Policy-based methods, in contrast, directly optimize the policy function and are often better suited for environments with continuous or high-dimensional action spaces.

Among modern policy-based algorithms, Proximal Policy Optimization (PPO) has gained widespread adoption due to its stability and sample efficiency. PPO constrains policy updates to prevent overly large changes, thereby improving training stability. These characteristics make PPO particularly attractive for network defense scenarios, where unstable policies may lead to disruptive or overly aggressive mitigation actions.

Overall, reinforcement learning provides a flexible and powerful framework for autonomous decision-making. Its ability to learn from interaction and adapt to changing environments forms the theoretical foundation for the AI-based network defense agent proposed in this study [8, 9].

2.2 Network Simulation Environments

Effective training and evaluation of reinforcement learning agents in cybersecurity require controlled, reproducible, and observable environments. Deploying learning agents directly

in production networks poses significant risks, including service disruption and unintended security violations. As a result, network simulation environments have become an essential research tool for developing and validating autonomous defense mechanisms.

2.2.1 Role of Simulation in Cybersecurity Research

Network simulations enable researchers to model complex infrastructures, generate realistic attack traffic, and observe system behavior under adversarial conditions. Unlike static datasets, simulated environments support interactive learning, where an agent's actions directly influence future network states. This characteristic aligns naturally with reinforcement learning, which depends on continuous agent–environment interaction.

Simulation-based approaches also allow the safe execution of destructive attack scenarios such as Distributed Denial-of-Service (DDoS), brute-force authentication attacks, and reconnaissance scanning. By isolating these activities from real-world systems, researchers can evaluate defense strategies without ethical or operational concerns. Moreover, simulation environments facilitate repeatable experiments, enabling fair comparisons between different learning algorithms and defense policies.

2.2.2 Mininet for Network Emulation

Mininet is a widely adopted network emulation framework that allows the creation of virtual networks using lightweight Linux containers. It provides realistic network behavior by leveraging the Linux kernel's networking stack, enabling accurate modeling of latency, bandwidth, and packet loss. This level of fidelity makes Mininet suitable for simulating enterprise-scale network topologies, including routers, switches, servers, and end hosts.

In cybersecurity research, Mininet has been extensively used to study intrusion detection systems, traffic analysis, and automated defense mechanisms. Its ability to dynamically reconfigure network topologies during runtime allows researchers to simulate evolving attack surfaces and defensive responses. Furthermore, Mininet integrates seamlessly with security tools such as Iptables, TC (Traffic Control), and packet inspection frameworks, enabling low-level enforcement of defensive actions.

2.2.3 Reinforcement Learning Environments with Gymnasium

Gymnasium, the successor of OpenAI Gym, provides a standardized interface for reinforcement learning environments. It defines clear abstractions for state spaces, action spaces, reward functions, and episode termination conditions. These abstractions simplify the integration of complex environments with reinforcement learning algorithms and training frameworks.

In the context of network defense, Gymnasium serves as the bridge between the reinforcement learning agent and the simulated network. Network metrics such as traffic volume,

connection rates, packet entropy, and alert signals can be encoded into numerical state representations. Similarly, defensive actions such as IP blocking, rate limiting, or traffic redirection can be mapped into discrete or continuous action spaces.

The use of Gymnasium ensures compatibility with widely adopted reinforcement learning libraries, including Stable Baselines3. This compatibility allows rapid experimentation with modern deep reinforcement learning algorithms while maintaining reproducibility and modularity.

2.2.4 Hybrid Simulation Architectures

Recent research increasingly adopts hybrid simulation architectures that combine network emulation platforms with reinforcement learning frameworks. In such architectures, Mininet is responsible for generating realistic network behavior, while Gymnasium manages the reinforcement learning loop. This separation of concerns enables flexible experimentation and simplifies system maintenance.

Hybrid environments also support the integration of monitoring and visualization tools, such as Security Information and Event Management (SIEM) systems. By feeding simulated network logs into SIEM platforms, researchers can observe how autonomous agents influence security metrics over time. This approach closely resembles real-world operational settings, enhancing the practical relevance of experimental results.

Overall, network simulation environments form the experimental backbone of autonomous cybersecurity research. By combining realistic network emulation with standardized reinforcement learning interfaces, these environments enable the safe and effective development of adaptive defense agents [10, 11].

2.3 Automated Incident Response Mechanisms

As cyber attacks become increasingly fast, scalable, and automated, traditional manual incident response processes are no longer sufficient. Security Operation Centers (SOCs) often face delayed response times due to alert overload, human error, and limited situational awareness. Automated Incident Response (AIR) mechanisms aim to address these challenges by enabling rapid, consistent, and adaptive defensive actions.

2.3.1 From Detection to Response Automation

Conventional security systems primarily focus on detection, leaving response decisions to human operators. While Intrusion Detection Systems (IDS) can identify suspicious activities, the lack of automated enforcement allows attackers to continue exploiting systems during response delays. Recent research emphasizes the necessity of coupling detection mechanisms with automated response to minimize the attack dwell time.

Automation enables immediate execution of predefined or adaptive actions once malicious behavior is identified. These actions include blocking malicious IP addresses, throttling abnormal traffic, isolating compromised hosts, or redirecting attackers to deception environments. By reducing reliance on manual intervention, automated response systems significantly improve reaction speed and operational consistency.

2.3.2 Rule-Based and Policy-Based Response Systems

Early automated response systems relied on static rules and predefined security policies. For example, if a connection threshold is exceeded, an IP address may be automatically blacklisted. While effective against known attack patterns, such rule-based approaches struggle to handle evolving threats and often generate false positives.

Policy-based systems attempt to improve flexibility by defining higher-level response objectives. However, these systems still depend heavily on expert-defined policies and lack the ability to learn from environmental feedback. As attack strategies become more adaptive, static response logic increasingly fails to maintain an optimal balance between security and service availability.

2.3.3 Reinforcement Learning for Adaptive Incident Response

Reinforcement Learning (RL) introduces a paradigm shift in automated incident response by enabling agents to learn optimal defense strategies through interaction with the environment. Instead of relying on fixed rules, RL agents evaluate the consequences of their actions using reward signals derived from security and performance metrics. This approach allows the system to dynamically adapt response strategies to different attack intensities and patterns.

In network defense scenarios, RL agents can autonomously select actions such as blocking, rate limiting, or monitoring based on the observed network state. Over time, the agent learns to minimize security risks while avoiding unnecessary disruptions to legitimate traffic. Several studies demonstrate that RL-based response systems outperform static approaches in handling complex, multi-stage attacks [6, 12].

2.3.4 Low-Level Enforcement Mechanisms

Effective automated response requires reliable enforcement mechanisms at the network and system levels. Linux kernel-level tools such as Iptables and Traffic Control (TC) are commonly used to implement real-time defensive actions. Iptables enables fine-grained packet filtering and connection blocking, while TC allows dynamic traffic shaping and rate limiting.

These mechanisms operate at a low level, ensuring minimal latency and high enforcement accuracy. By integrating RL decision-making with kernel-level controls, automated defense

systems can respond to attacks with both intelligence and speed. Such integration bridges the gap between high-level AI reasoning and practical network security enforcement.

2.3.5 Deception-Based Response and Honeypots

Deception techniques play an increasingly important role in modern incident response strategies. Honeypots are decoy systems designed to attract attackers and observe malicious behavior without risking production assets. By redirecting suspicious traffic to honeypots, defenders can gather valuable intelligence while reducing direct attack impact.

Automated redirection mechanisms allow deception to be applied dynamically, based on real-time threat assessments. When combined with RL, deception strategies can be selectively activated to maximize intelligence gain while minimizing system disruption. This adaptive use of honeypots enhances both defensive effectiveness and situational awareness.

2.3.6 Integration with SIEM Systems

Security Information and Event Management (SIEM) systems aggregate and correlate security logs from multiple sources to provide centralized visibility. Integrating automated response mechanisms with SIEM platforms allows administrators to monitor defensive actions and system performance in real time. Visualization and alert correlation improve trust in autonomous systems by providing transparency and auditability.

In research environments, SIEM integration enables quantitative evaluation of response effectiveness through metrics such as alert reduction, response latency, and attack mitigation success. These insights are critical for validating autonomous defense agents and assessing their readiness for real-world deployment [13, 14].

2.4 Feature Engineering for Network Traffic

Feature engineering plays a critical role in applying machine learning and reinforcement learning techniques to network security. Raw network traffic, consisting of packets and flows, is inherently high-dimensional, noisy, and heterogeneous. Without appropriate feature representation, learning agents may struggle to distinguish between benign and malicious behaviors.

2.4.1 Packet-Level and Flow-Level Features

Network traffic features can generally be categorized into packet-level and flow-level representations. Packet-level features include attributes such as source and destination IP addresses, protocol types, packet sizes, and TCP flags. While packet-level data provides

fine-grained visibility, it often results in high computational overhead and limited temporal context.

Flow-level features aggregate packets sharing common characteristics within a time window. Common flow attributes include connection duration, number of packets, byte counts, average packet size, and inter-arrival times. Flow-based representations reduce data dimensionality and better capture behavioral patterns, making them more suitable for real-time analysis.

2.4.2 Statistical and Behavioral Features

Beyond basic header information, statistical features provide insights into traffic dynamics. Metrics such as packet rate, connection frequency, entropy of source or destination addresses, and variance of packet sizes are commonly used. These features are effective in detecting volumetric attacks such as Denial-of-Service (DoS) and Distributed Denial-of-Service (DDoS).

Behavioral features capture deviations from normal traffic patterns over time. Examples include sudden spikes in connection attempts, abnormal protocol usage, and repeated authentication failures. Such features are particularly valuable for identifying reconnaissance activities and brute-force attacks.

2.4.3 Feature Extraction Tools and Techniques

Automated feature extraction tools are essential for real-time network monitoring. Scapy is a widely used Python-based framework for packet capturing and protocol analysis. It enables flexible parsing of packet headers and payloads, allowing custom feature extraction pipelines to be implemented.

In simulated environments, packet capture can be combined with flow aggregation techniques to generate structured feature vectors. These vectors serve as numerical representations of the network state and are suitable as inputs for reinforcement learning agents. Efficient feature extraction ensures that state observations accurately reflect the current security posture of the network.

2.4.4 State Representation for Reinforcement Learning

In reinforcement learning-based defense systems, feature engineering directly influences the design of the state space. The state representation must balance informativeness with computational efficiency. Overly complex states may slow training and cause instability, while overly simplistic states may omit critical security signals.

Typical state vectors may include normalized traffic statistics, alert counts, and resource utilization metrics. By encoding both security-related and performance-related features,

the agent can learn policies that balance attack mitigation with service availability. This holistic state representation is essential for autonomous decision-making in dynamic network environments.

2.4.5 Feature Set for Defense Agent

This research employs a comprehensive set of 12 engineered features designed to capture both network-level and application-level attack indicators. Each feature is normalized to the range $[0, 1]$ to facilitate stable reinforcement learning training. The features are categorized into network traffic features and application layer features.

Network Traffic Features

- **Feature 1 - Packet Rate:** Measures the average packet rate per second, calculated as $v = \min(\text{count}/1000, 1.0)$. A threshold of > 1200 packets per second indicates potential volumetric DoS/DDoS attacks [15, 16].
- **Feature 2 - SYN Ratio:** Computes the ratio of SYN packets to ACK packets using $v = \min((\text{SYN}/\max(\text{ACK}, 1))/2.0, 1.0)$. A ratio exceeding 2.0 suggests SYN flood attacks with incomplete handshakes [17, 18].
- **Feature 3 - Distinct Port Count:** Tracks unique destination ports accessed within a time window via $v = \min(\text{unique\_dst\_ports}/50, 1.0)$. More than 20 ports accessed within 60 seconds triggers port scanning alerts [19].
- **Feature 4 - RST Ratio:** Measures the proportion of RST (reset) packets to total packets as $v = \text{RST\_count}/\max(\text{Total\_pkts}, 1)$. A ratio exceeding 0.20 indicates potential backscatter from scanning activities [20].
- **Feature 5 - Payload Length:** Normalizes payload size against the standard MTU using $v = \min(\text{len(payload)}/1500, 1.0)$. Payloads exceeding 1500 bytes may indicate fragmentation attacks [21, 22].

Application Layer Features

- **Feature 6 - HTTP Error Ratio:** Calculates the proportion of 4xx and 5xx HTTP responses as $v = \text{count}(4xx, 5xx)/\text{total\_responses}$. A ratio above 0.25 suggests fuzzing or enumeration attempts [23].
- **Feature 7 - SQL Injection Signature:** Detects SQL injection patterns using regex matching with $v = \min(\text{count(Regex\_SQL)}/3, 1.0)$. Any match triggers critical alerts based on OWASP ModSecurity Core Rule Set [24].
- **Feature 8 - XSS Signature:** Identifies cross-site scripting patterns via $v = \min(\text{count(Regex\_XSS)}/3, 1.0)$, detecting tags such as `<script>` and event handlers [24].

- **Feature 9 - SQL Special Character Ratio:** Measures the density of SQL syntax characters ($'$, $-$, ;, #) using $v = \text{count}([\text{'}-;\#])/\text{len}(\text{payload})$. Ratios above 0.08 indicate obfuscated SQLi attempts [25].
- **Feature 10 - XSS Special Character Ratio:** Computes the density of HTML/JavaScript characters ($<$, $>$, /, ( ), $=$) as $v = \text{count}([<> /() =])/\text{len}(\text{payload})$. Ratios exceeding 0.15 suggest XSS payloads [25].
- **Feature 11 - Percent Encoding Ratio:** Detects URL encoding evasion by calculating $v = \text{count}(\text{'\%'})/\text{len}(\text{payload})$. Ratios above 0.30 indicate double URL encoding attacks [26].
- **Feature 12 - Numeric Ratio:** Measures digit density via $v = \text{count}(\text{digits})/\text{len}(\text{payload})$ to detect hex injection or integer-based SQLi. Ratios above 0.40 in text fields are classified as malicious [27].

These features are extracted in real-time from network packets and HTTP logs using Scapy and custom parsing modules. The thresholds are derived from established security research and validated against benchmark datasets including CIC-IDS2017 and UNSW-NB15.

2.4.6 Challenges in Network Feature Engineering

Feature engineering for network traffic presents several challenges. Encrypted traffic limits visibility into payload contents, requiring reliance on metadata and statistical features. Additionally, network behavior is highly dynamic, leading to concept drift that can degrade model performance over time.

Another challenge lies in feature selection and normalization. Redundant or highly correlated features may negatively impact learning efficiency. Therefore, careful feature design and continuous evaluation are necessary to maintain robust and adaptive defense performance [28, 29].

2.5 Summary and Research Gap

This chapter reviewed the existing literature related to network security, reinforcement learning, automated incident response, and network traffic feature engineering. Traditional network defense mechanisms, such as rule-based firewalls and signature-based intrusion detection systems, remain effective against known attack patterns but struggle to adapt to novel and automated threats. The increasing scale and sophistication of cyber attacks have exposed the limitations of manual and static defense strategies.

Recent advances in reinforcement learning demonstrate strong potential for adaptive and autonomous network defense. RL-based approaches enable systems to learn optimal re-

sponse policies through interaction with dynamic environments, offering advantages over fixed-rule and policy-driven mechanisms. Research has shown that RL agents can effectively mitigate various attack types while considering system performance constraints.

Network simulation environments, particularly those combining realistic emulation platforms with standardized RL interfaces, have become essential tools for cybersecurity research. Mininet enables high-fidelity modeling of network behavior, while Gymnasium provides a structured framework for agent–environment interaction. These environments allow safe, repeatable experimentation and facilitate the evaluation of autonomous defense strategies under controlled conditions.

Despite these advancements, several research gaps remain. Many existing studies focus primarily on attack detection rather than closed-loop defense systems that integrate detection, decision-making, and enforcement. Furthermore, a significant portion of prior work evaluates defense effectiveness solely based on security metrics, without adequately considering the impact on legitimate traffic and service availability.

Another limitation lies in the integration of learning agents with real-world enforcement mechanisms. While theoretical models demonstrate promising results, fewer studies implement low-level controls such as kernel-level packet filtering or traffic shaping. Additionally, the use of deception techniques, such as adaptive honeypots, is often treated as a static component rather than a dynamically learned response.

This research addresses these gaps by proposing an autonomous AI defense agent that operates in a closed-loop simulation environment. The system integrates reinforcement learning with realistic network emulation, low-level enforcement mechanisms, and deception-based defense strategies. By incorporating both security and performance metrics into the learning process, the proposed approach aims to achieve a balanced and practical solution for adaptive network defense.

The next chapter presents the system architecture and methodology, detailing the design of the simulation environment, the reinforcement learning framework, and the implementation of automated response mechanisms.

Bibliography

[1] Verizon, "2025 data breach investigations report," Verizon Communications Inc., Tech. Rep., 2025, accessed: January 2026. [Online]. Available: https://www.verizon.com/business/resources/reports/dbir/

[2] ——, "2024 data breach investigations report," Verizon Communications Inc., Tech. Rep., 2024, accessed: January 2026. [Online]. Available: https://www.verizon.com/business/resources/reports/2024-dbir-data-breach-investigations-report.pdf

[3] ——, "2020 data breach investigations report," Verizon Communications Inc., Tech. Rep., 2020, accessed: January 2026. [Online]. Available: https://www.verizon.com/business/resources/reports/2020-data-breach-investigations-report.pdf

[4] IBM Security and Ponemon Institute, "Cost of a data breach report 2024," IBM Corporation, Tech. Rep., 2024, free registration required. Executive summary available without registration. [Online]. Available: https://www.ibm.com/reports/data-breach

[5] B. A. Alahmadi, L. Axon, and I. Martinovic, "99% false positives: A qualitative study of soc analysts' perspectives on security alarms," in *Proceedings of the 31st USENIX Security Symposium*, 2022. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/alahmadi

[6] T. T. Nguyen and V. J. Reddi, "Deep reinforcement learning for cyber security," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 8, pp. 3779–3795, 2021, open access arXiv: https://arxiv.org/abs/1906.05799.

[7] MITRE Corporation, "Mitre att&ck framework," Online, 2023, accessed: January 2026. [Online]. Available: https://attack.mitre.org/

[8] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018, free PDF available from author's website. [Online]. Available: http://incompleteideas.net/book/the-book-2nd.html

[9] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017. [Online]. Available: https://arxiv.org/abs/1707.06347

[10] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: Rapid prototyping for software-defined networks," in *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks*, 2010. [Online]. Available: http://mininet.org/papers/mininet-hotnets2010.pdf

[11] Farama Foundation, "Gymnasium: A standard interface for reinforcement learning environments," Online Documentation, 2023. [Online]. Available: https://gymnasium.farama.org/

[12] M. Lopez-Martin, B. Carro, and A. Sanchez-Esguevillas, "Application of deep reinforcement learning to intrusion detection for supervised problems," *Expert Systems with Applications*, vol. 141, 2020, open access arXiv: https://arxiv.org/abs/1906.09478.

[13] IBM Security, "What is soar (security orchestration, automation and response)?" IBM Documentation, 2023, accessed: January 2026. [Online]. Available: https://www.ibm.com/topics/soar

[14] M. F. M. Razali, M. F. A. R. Noor, and A. Zainal, "A survey of honeypots and honeynets for internet of things, industrial internet of things, and cyber-physical systems," *IEEE Access*, vol. 9, pp. 151 620–151 638, 2021, open access arXiv: https://arxiv.org/abs/2108.02287.

[15] National Institute of Standards and Technology, "Resilient interdomain traffic exchange: Bgp security and ddos mitigation," NIST, Tech. Rep. SP 800-189, 2019. [Online]. Available: https://csrc.nist.gov/publications/detail/sp/800-189/final

[16] R. Mahajan, D. Wetherall, and T. Anderson, "Understanding bgp misconfiguration," in *Proceedings of ACM SIGCOMM*, 2002, iMAP Framework for DDoS mitigation.

[17] W. Eddy, "Tcp syn flooding attacks and common mitigations," RFC 4987, 2007. [Online]. Available: https://tools.ietf.org/html/rfc4987

[18] H. Wang, D. Zhang, and K. G. Shin, "Detecting syn flooding attacks," in *Proceedings of IEEE INFOCOM*, 2002.

[19] J. Jung, V. Paxson, A. W. Berger, and H. Balakrishnan, "Fast portscan detection using sequential hypothesis testing," in *Proceedings of IEEE Symposium on Security and Privacy*, 2004.

[20] V. Paxson, "Bro: A system for detecting network intruders in real-time," in *Proceedings of the 7th USENIX Security Symposium*, 1999, now known as Zeek IDS.

[21] C. Hornig, "A standard for the transmission of ip datagrams over ethernet networks," RFC 894, 1984. [Online]. Available: https://tools.ietf.org/html/rfc894

[22] J. Postel, "Internet protocol," RFC 791, 1981. [Online]. Available: https://tools.ietf.org/html/rfc791

[23] OWASP Foundation, "Owasp appsensor project," Online, 2023. [Online]. Available: https://owasp.org/www-project-appsensor/

[24] ——, "Owasp modsecurity core rule set (crs) v3.x," Online, 2023. [Online]. Available: https://coreruleset.org/

[25] C. Kruegel and G. Vigna, "Anomaly detection of web-based attacks," in *Proceedings of the 10th ACM Conference on Computer and Communications Security (CCS)*, 2003.

[26] T. Berners-Lee, R. Fielding, and L. Masinter, "Uniform resource identifier (uri): Generic syntax," RFC 3986, 2005. [Online]. Available: https://tools.ietf.org/html/rfc3986

[27] R. Kozik, M. Choraś, R. Renk, and W. Holubowicz, "Cyber security of web applications: Current state and roadmap for future research," *International Joint Conference SOCO-CISIS-ICEUTE*, 2015.

[28] P. Biondi, "Scapy: Packet manipulation tool," Online, 2007. [Online]. Available: https://scapy.net/

[29] M. A. Ferrag, L. A. Maglaras, and H. Janicke, "Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study," *Journal of Information Security and Applications*, vol. 50, 2020, arXiv preprint available at: https://arxiv.org/abs/1904.02057.